

最优化理论大作业实验报告

问题描述

使用邻近梯度法、交替方向乘子法，次梯度法求解一个十节点的分布式系统的一范数正则化最小二乘问题。

模型如下：

$$\min_{\mathbf{x}} \quad \frac{1}{2} \|A_1 \mathbf{x} - \mathbf{b}_1\|_2^2 + \cdots + \frac{1}{2} \|A_{10} \mathbf{x} - \mathbf{b}_{10}\|_2^2 + \lambda \|\mathbf{x}\|_1,$$

每个节点有线性测量：

$$\mathbf{b}_i = A_i \mathbf{x} + \mathbf{e}_i, \quad i = 1, \dots, 10$$

其中：

- λ 是正则化参数， $\lambda > 0$ 。
- $\mathbf{x}$ 是200维的未知稀疏向量，稀疏度为5。
- $\mathbf{x}$ 真值中的非零元素服从均值为0方差为1的高斯分布。
- A 是5 * 200 维的矩阵，元素服从均值为0方差为1的高斯分布。
- $\mathbf{b}$ 是5维的测量值。
- $\mathbf{e}_i$ 是5维的测量噪声，其中的元素服从均值为0方差为0.1的高斯分布。

算法设计

邻近梯度法

邻近梯度法用于求解以下形式的优化问题：

$$\min_{\mathbf{x}} f(\mathbf{x}) + \lambda g(\mathbf{x})$$

其中， $f(\mathbf{x})$ 是一个光滑的目标函数，可微； $g(\mathbf{x})$ 是一个非光滑的正则化项，通常不可微。

在优化 $f(\mathbf{x})$ 时，直接用梯度下降法计算梯度并沿着梯度的反方向更新。

对于 $g(\mathbf{x})$ ，通过临近点操作来更新。对于L1正则化，采取的临近点操作通常是软阈值操作。它的定义是：

$$\text{soft_threshold}(z, \lambda) = \text{sign}(z) \cdot \max(|z| - \lambda, 0)$$

实现邻近梯度法的步骤是：

1. 初始化 $\mathbf{x}^0$ 为零向量。
2. 迭代步骤
 1. 计算目标函数的梯度，用梯度下降更新参数 $\mathbf{x}$ 。
 2. 对更新后的参数应用软阈值操作，得到新的解 $\mathbf{x}^{k+1}$ 。

3. 检查收敛条件。当解收敛或迭代次数达到最大值时，算法终止。

交替方向乘子法

交替乘子法将原始的问题分解成多个子问题，交替地优化这些子问题，并使用拉格朗日乘子表示问题之间的联系。首先，固定正则化项和拉格朗日乘子，优化目标函数；随后，固定目标函数和拉格朗日乘子，优化正则化项；最后，固定目标函数和正则化项，更新拉格朗日乘子。

在这个问题中，实现交替方向乘子法的步骤是：

1. 初始化 x^0 为零向量。

2. 迭代步骤

1. 通过求解：

$$(A^T A + C \cdot I)x = A^T b + C \cdot y - v$$

来更新 x 。

2. 通过软阈值操作来更新 y （对 $x + v/C$ ）进行软阈值操作。

3. 通过 $v = v + C \cdot (x - y)$ 来更新 v 。

3. 检查收敛条件。当解收敛或迭代次数达到最大值时，算法终止。

次梯度法

次梯度法通常用于求解带有非光滑项的优化问题。对于不可分的部分，我们求它的次梯度，用它来进行梯度下降。对L1正则化问题，它的次梯度是元素的符号函数。对于零点，我们给它设置一个 $[-1, 1]$ 之间的随机值。

在这个问题中，实现次梯度法的步骤是：

1. 初始化 x^0 和次梯度向量。

2. 迭代步骤

1. 计算目标函数的梯度，计算正则化部分的次梯度，以它们的和作为用于梯度下降的次梯度。

2. 用梯度下降法更新 x 。

3. 检查收敛条件。当解收敛或迭代次数达到最大值时，算法终止。

数值实验

初始化

```

# 生成A: 均值为0方差为1的高斯分布
A = np.array([np.random.normal(loc=0, scale=1, size=(5, 200))for _ in range(10)])
# 生成x的真值
# 200维
x_true = np.zeros(200)
# 稀疏度为5
nonzero = np.random.choice(a=200, size=5, replace=False)
# 非零元素服从均值为0方差为1的高斯分布
x_true[nonzero] = np.random.normal(loc=0, scale=1, size=5)
# 生成b: b=Ai x + ei
# ei为5维的测量噪声, 服从均值为0方差为0.1的高斯分布。
b = np.array([A[i].dot(x_true) + np.random.normal(loc=0, scale=0.1, size=5) for i in range(10)])

# 求解
type = 2
lamda = 0.01
if type == 0:
    proximal_gradient_method(A, b, lamda, x_true)
elif type == 1:
    admm(A, b, lamda, x_true)
elif type == 2:
    subgradient(A, b, lamda, x_true)

```

设置A, x的真值, b, lamda, 选择优化算法。

邻近梯度法

```

# 初始化解的数组
solutions = []
# 初始化解, X是200维的变量
x = np.zeros(200)
# 学习率
learning_rate = 0.0001
for _ in range(3000):
    cur_x = x.copy()
    # 计算梯度
    gradient = np.zeros(200)
    for i in range(10):
        gradient = gradient + A[i].T.dot(A[i].dot(x) - b[i])
    x = (np.sign(x - learning_rate * gradient) *
        np.maximum(np.abs(x - learning_rate * gradient)
            - lamda * learning_rate, 0))
    solutions.append(x)

    if np.linalg.norm(x - cur_x, ord=2) < 1e-5:
        break

```

初始化 x^0 , 设置学习率和迭代次数, 开始迭代。

当迭代次数达到3000, 或者两次迭代之间X的差异小于 10^{-5} , 计算停止。

交替方向乘子法

```
results = []
start_time = time.time()
for _ in range(3000):
    x_cur = x.copy()

    # 更新x
    x = np.linalg.inv(np.sum([A[i].T.dot(A[i]) for i in range(10)],
                             axis=0) + C * np.eye((200)))
    x = x.dot(np.sum([A[i].T.dot(b[i]) for i in range(10)], axis=0)
              + C * y - v)

    # 更新y
    y = np.sign(x + v/C) * np.maximum(np.abs(x + v/C) - lamda / C, 0)
    results.append(x)

    # 判断收敛
    if np.linalg.norm(x - x_cur, ord=2) < 1e-5:
        break
```

初始化、收敛条件和邻近梯度法相同。

次梯度法

```
for _ in range(3000):
    x_cur = x.copy()

    # 次梯度
    g = np.empty_like(x)
    for i, data in enumerate(x):
        if data == 0:
            g[i] = 2 * np.random.random() - 1 # [-1, 1]
        else:
            g[i] = np.sign(x[i])
    g *= lamda
    g += np.sum([A[i].T.dot(A[i].dot(x) - b[i]) for i in range(10)],
                axis=0)

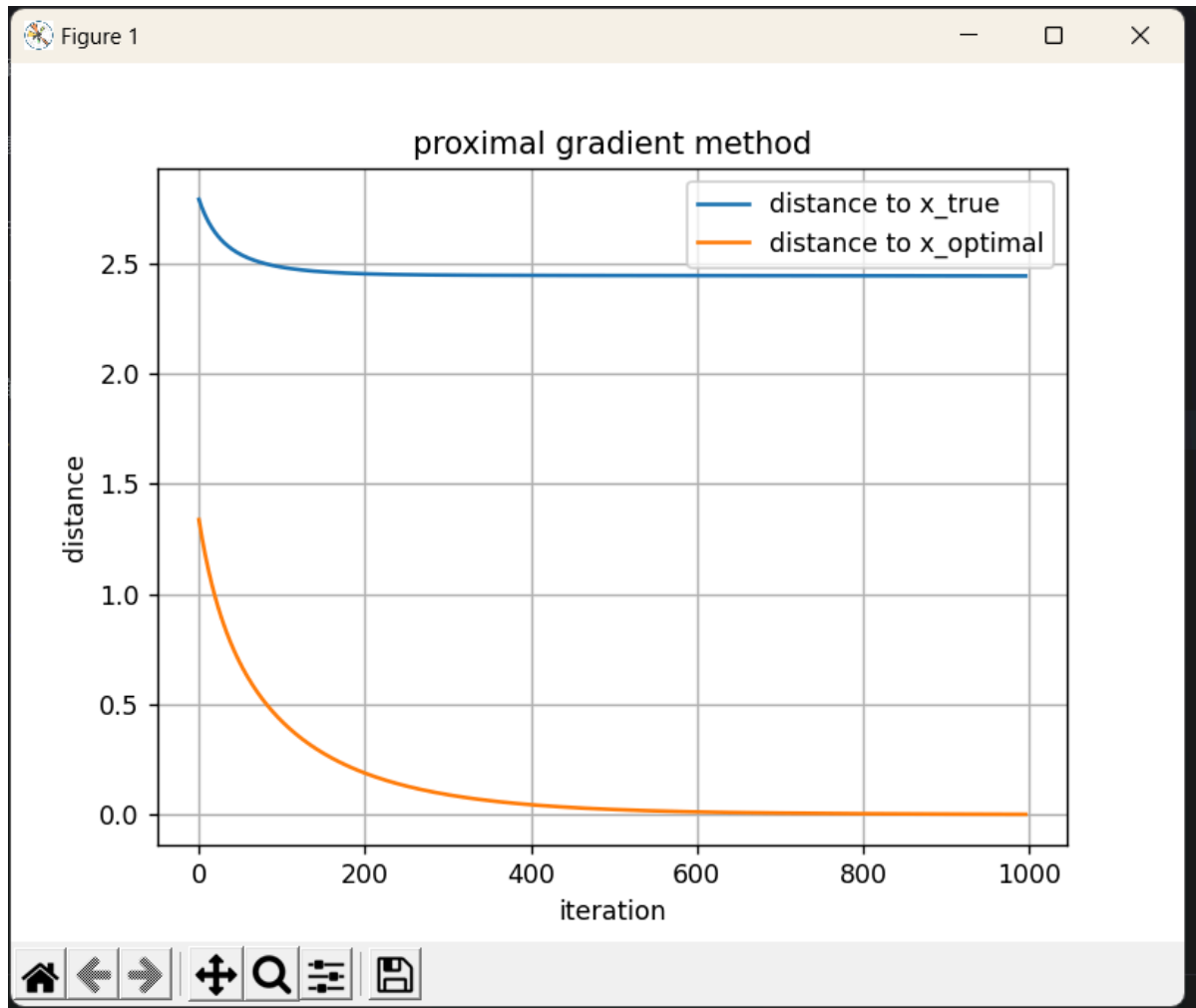
    # 更新x
    x = x - learning_rate * g

    results.append(x)

    # 判断收敛
    if np.linalg.norm(x - x_cur, ord=2) < 1e-5:
        break
```

结果分析

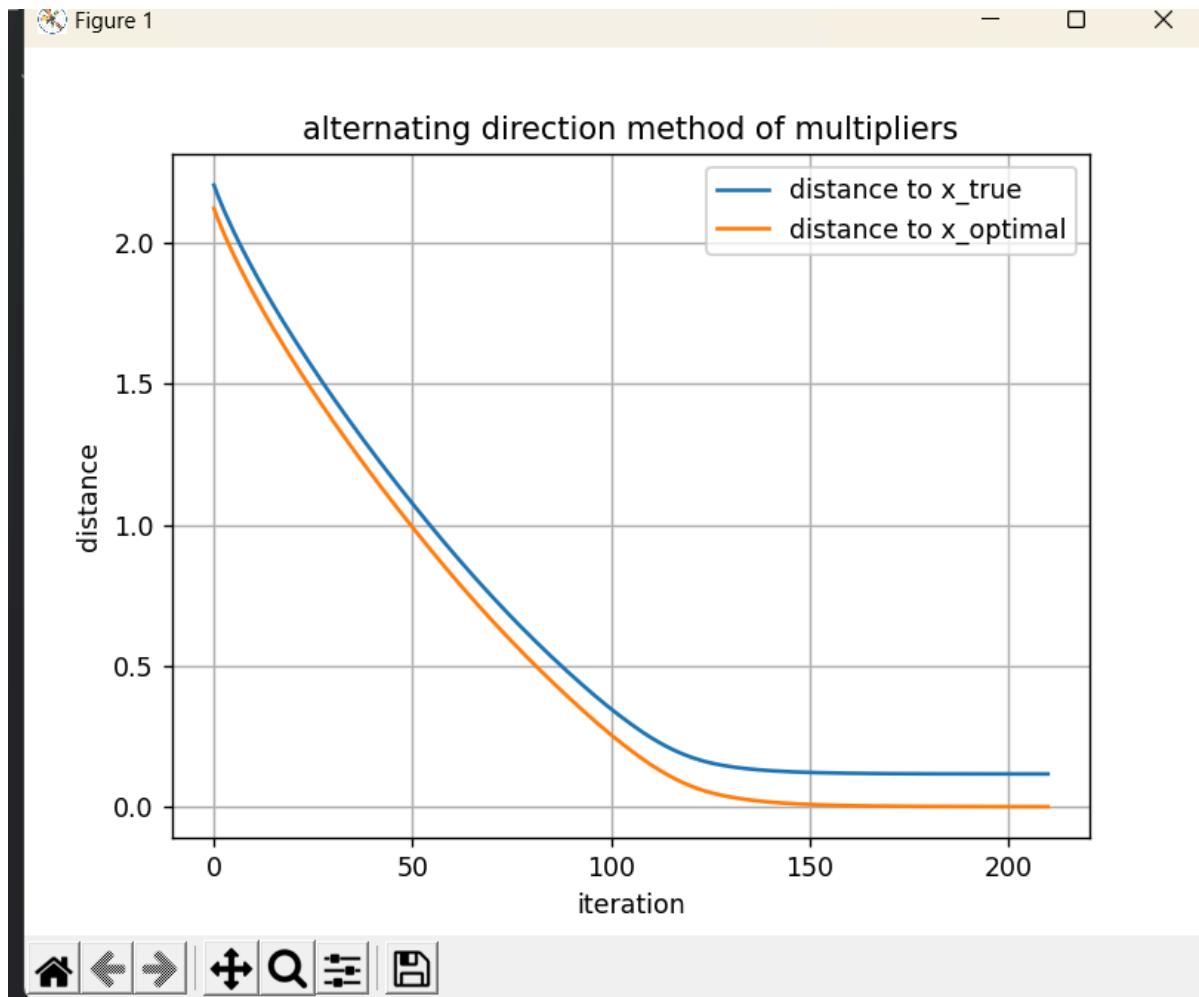
邻近梯度法



$\lambda = 0.01$ 时的结果。

迭代到1000次时收敛，到真值和最优解的距离分别收敛到2.5和0.

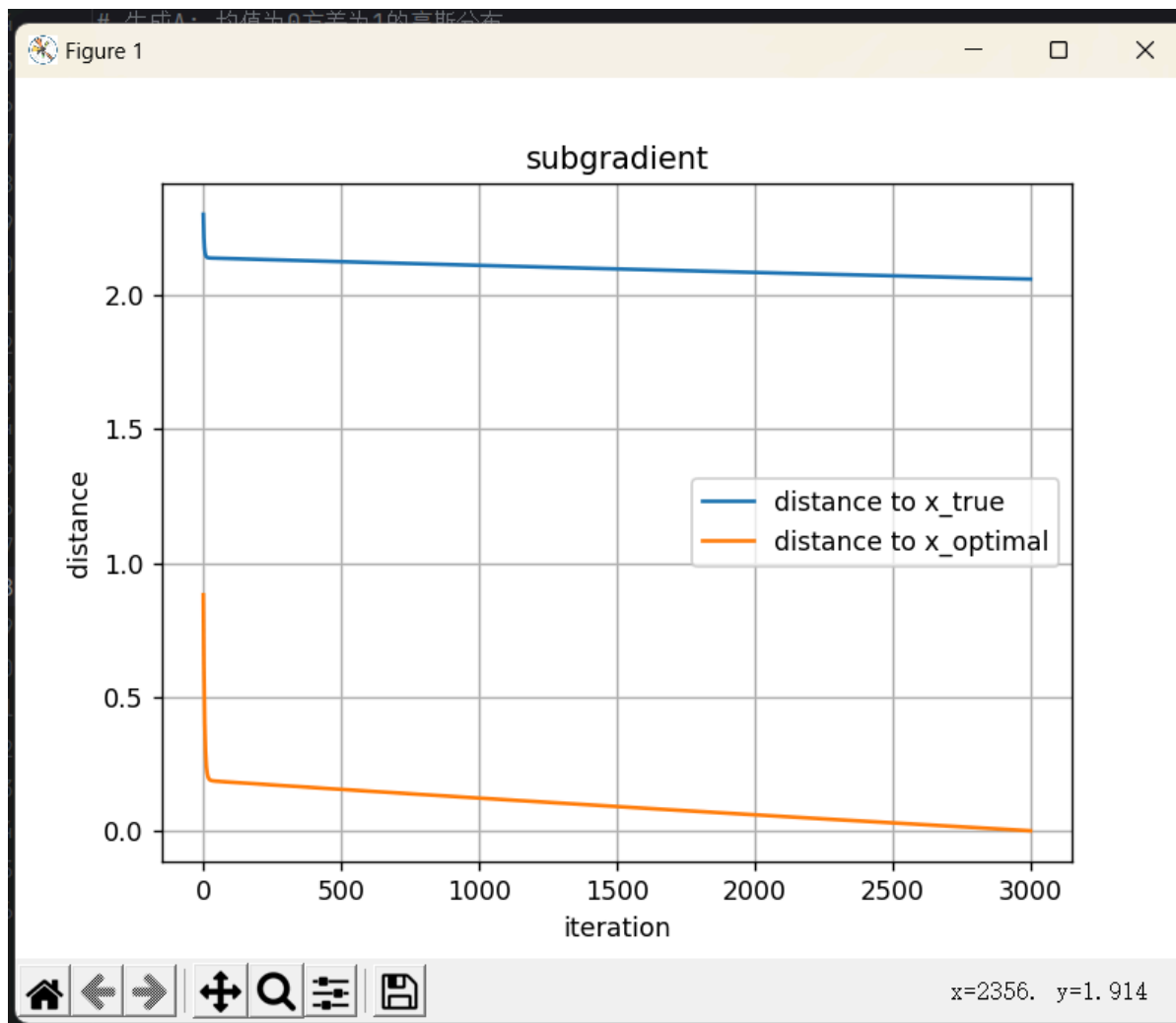
交替方向乘法



$\lambda = 0.01$.

迭代到200次时收敛，到真值和最优解的距离分别收敛到0.1和0。

次梯度法



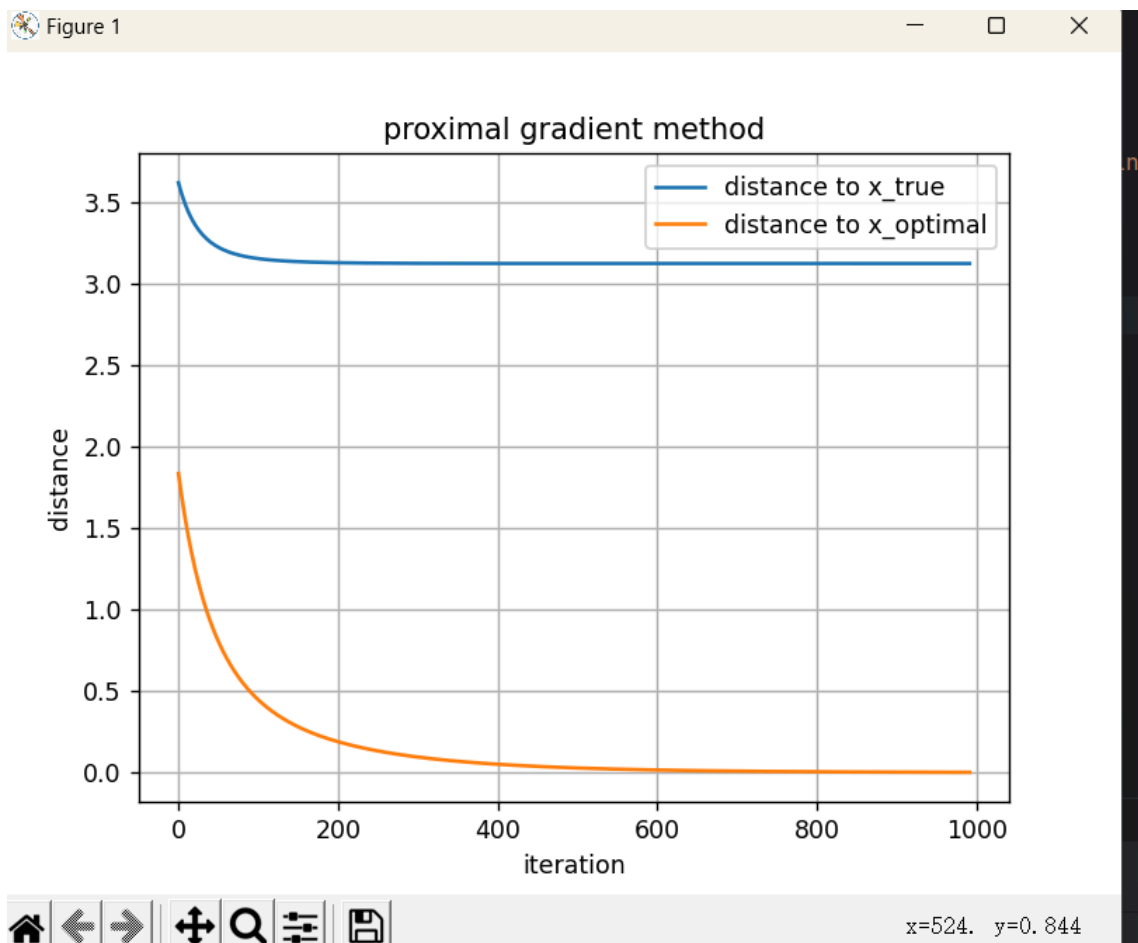
$\lambda = 0.01$.

此时没有收敛，最终到真值和最优解的距离分别接近2和0.

正则化参数对计算结果的影响

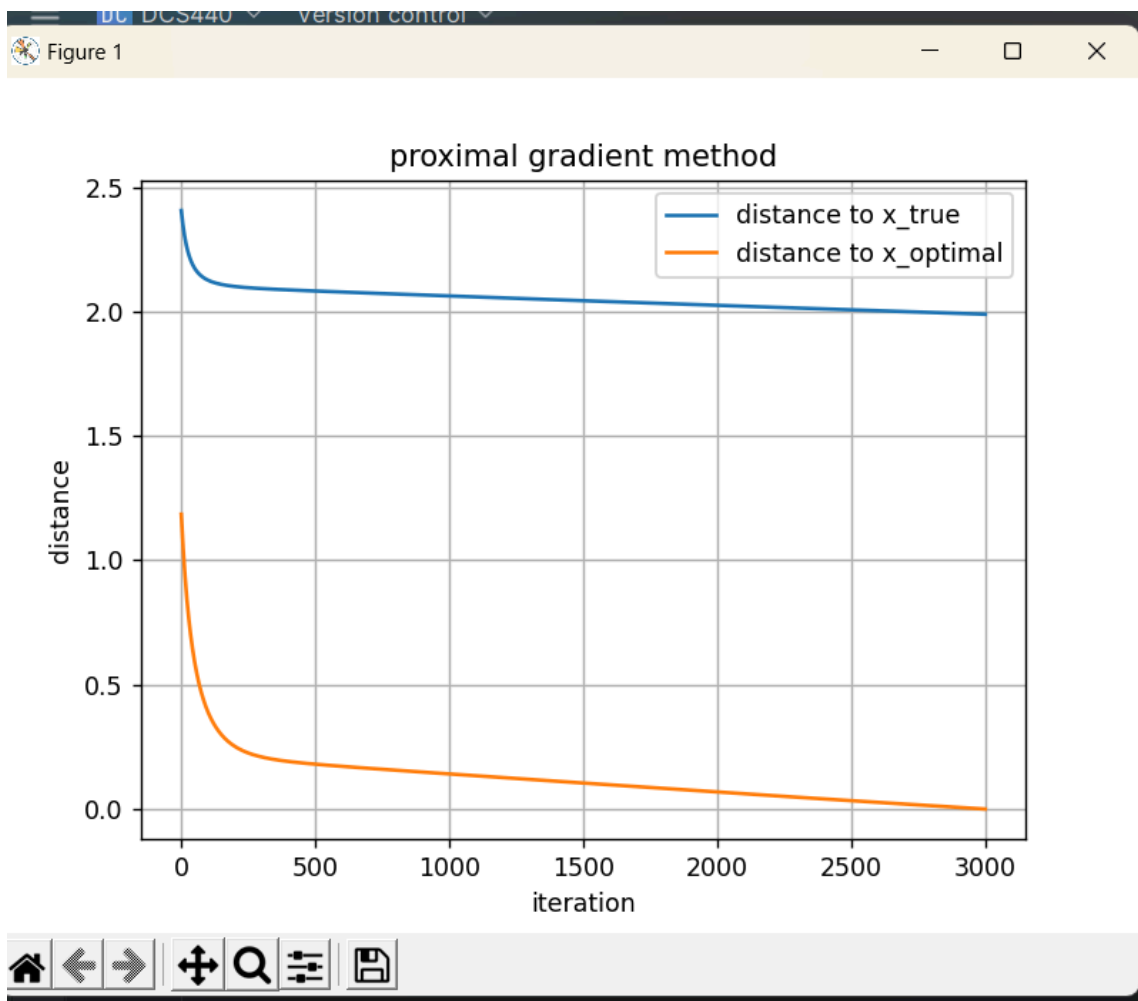
邻近梯度法

1. $\lambda = 0.001$



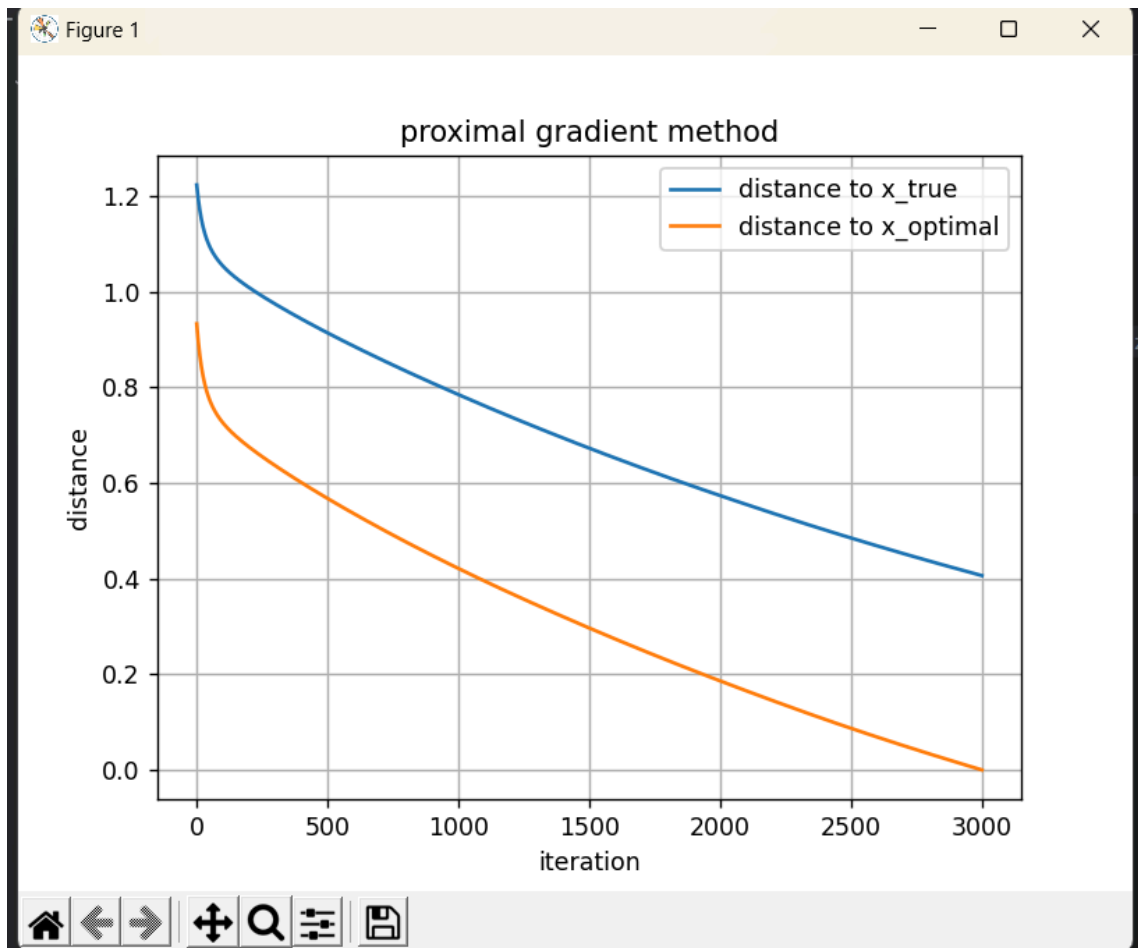
迭代到1000次时收敛，到真值和最优解的距离分别收敛到3.1和0.

2. $\lambda = 0.1$



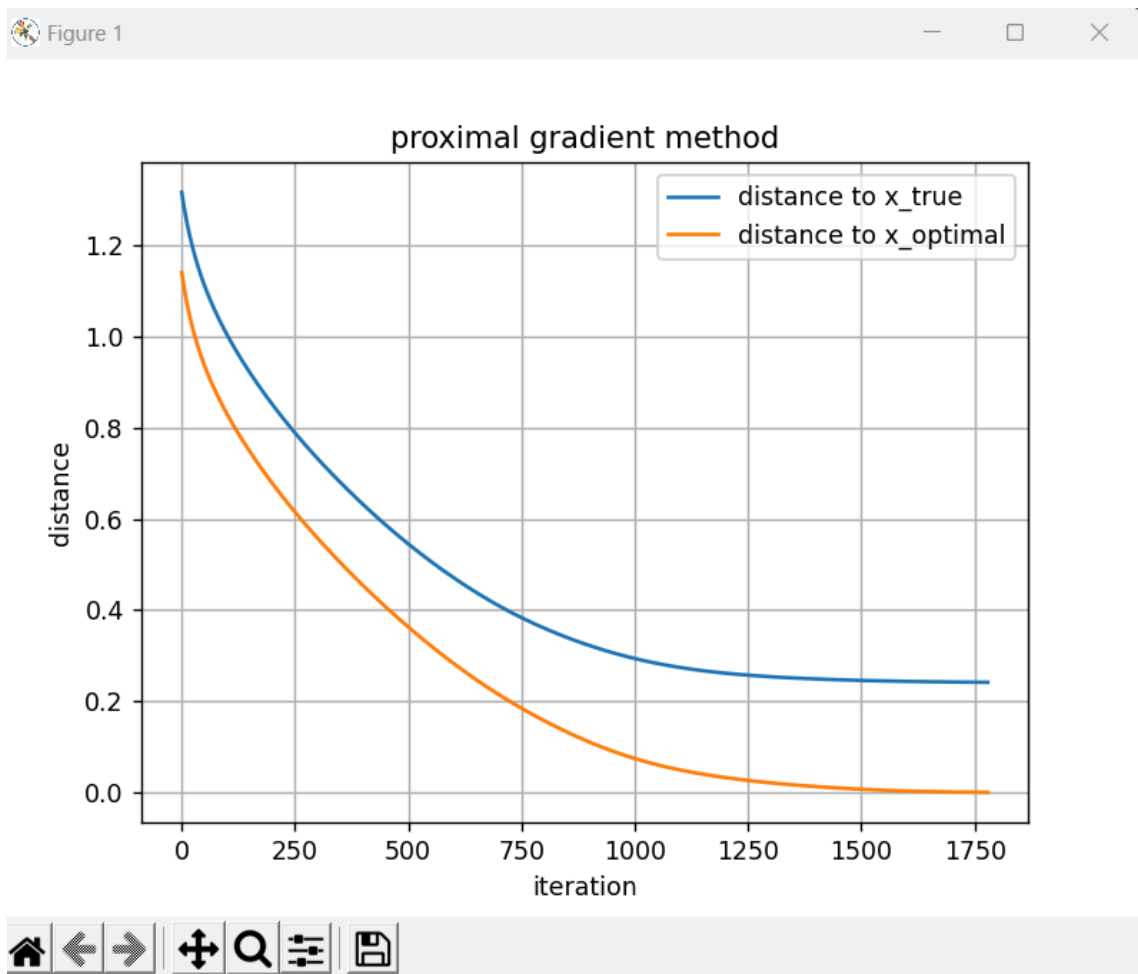
没有收敛，到真值和最优解的距离分别接近2.0和0.

3. $\lambda = 1.0$



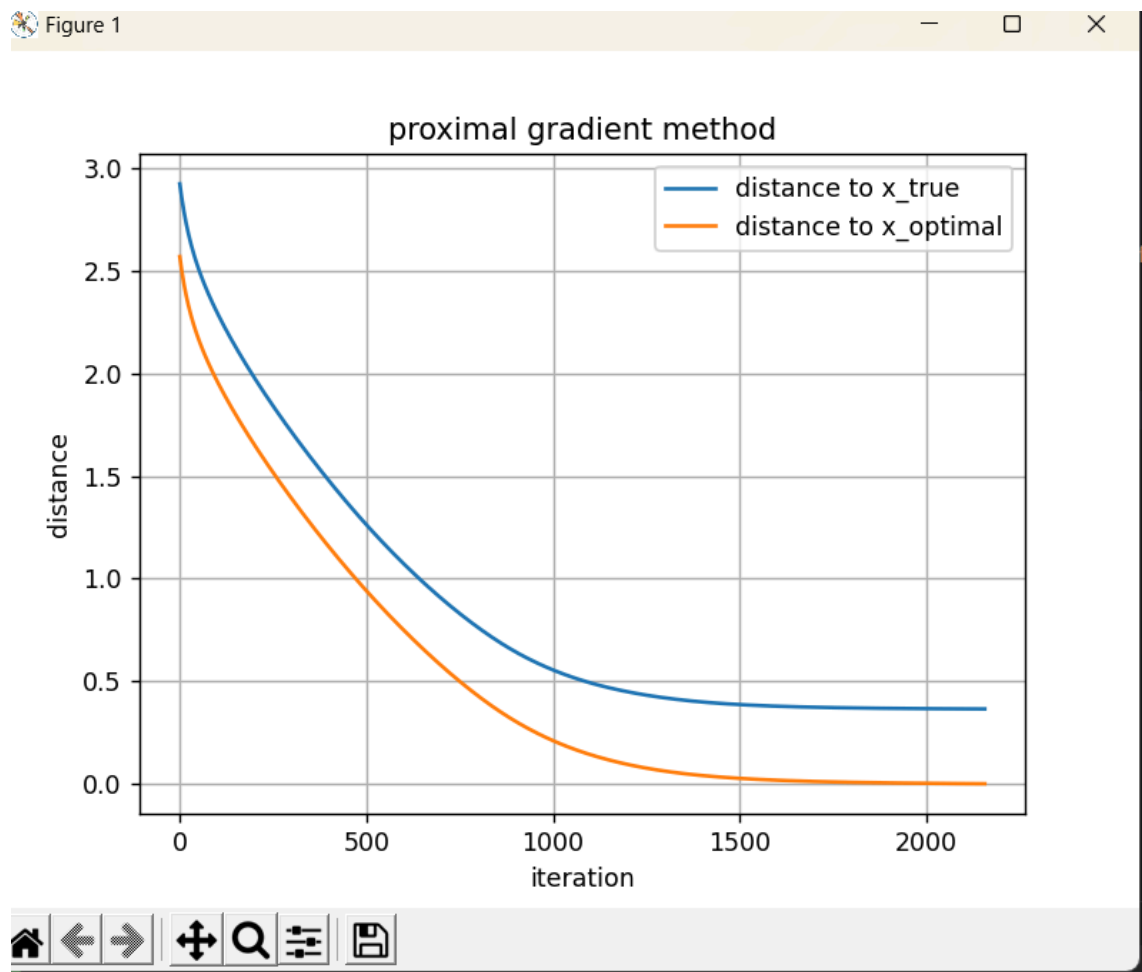
没有收敛，到真值和最优解的距离分别接近0.4和0.

4. $\lambda = 5.0$



迭代到1750次时收敛，到真值和最优解的距离分别收敛到0.25和0.

5. $\lambda = 10.0$

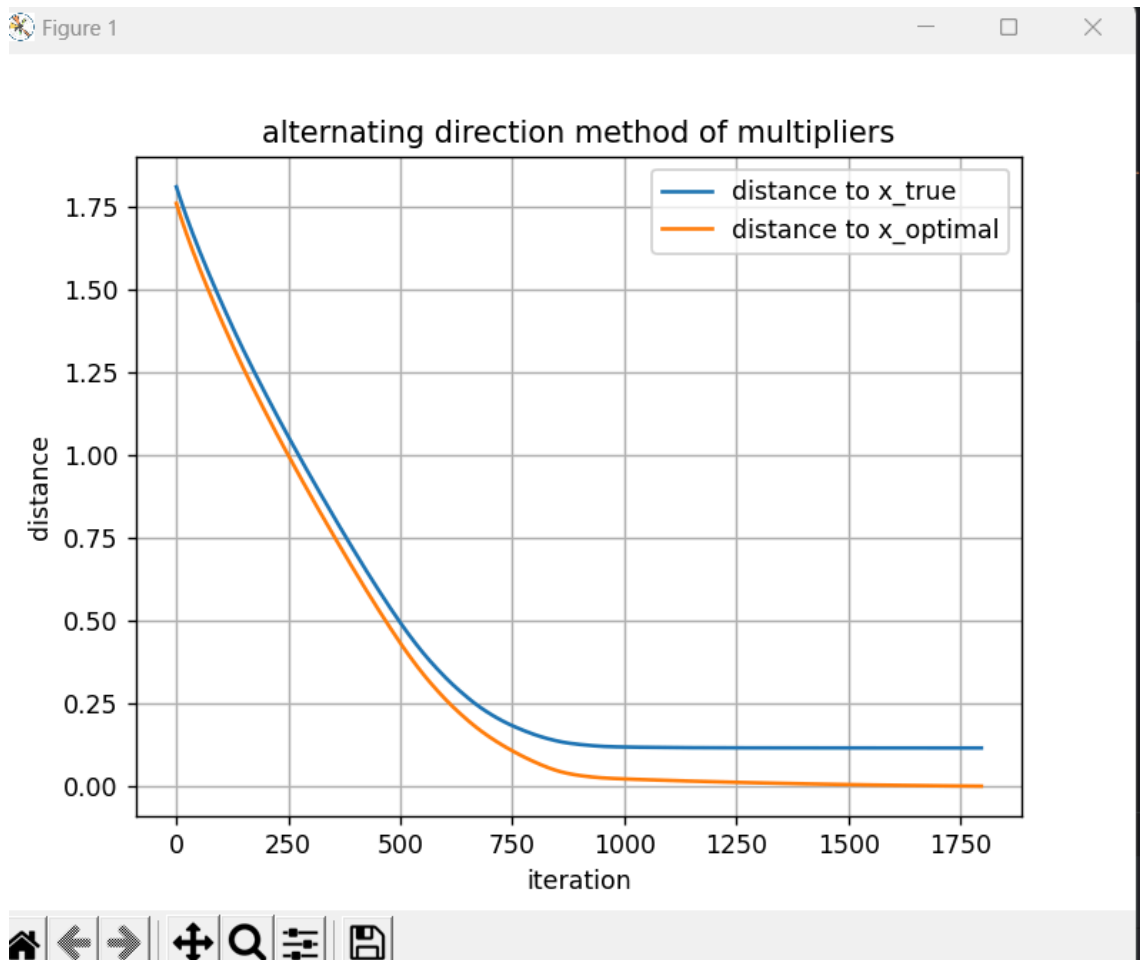


迭代到1750次时收敛，到真值和最优解的距离分别收敛到0.4和0.

对于邻近梯度法， λ 的最优值在5.0附近。

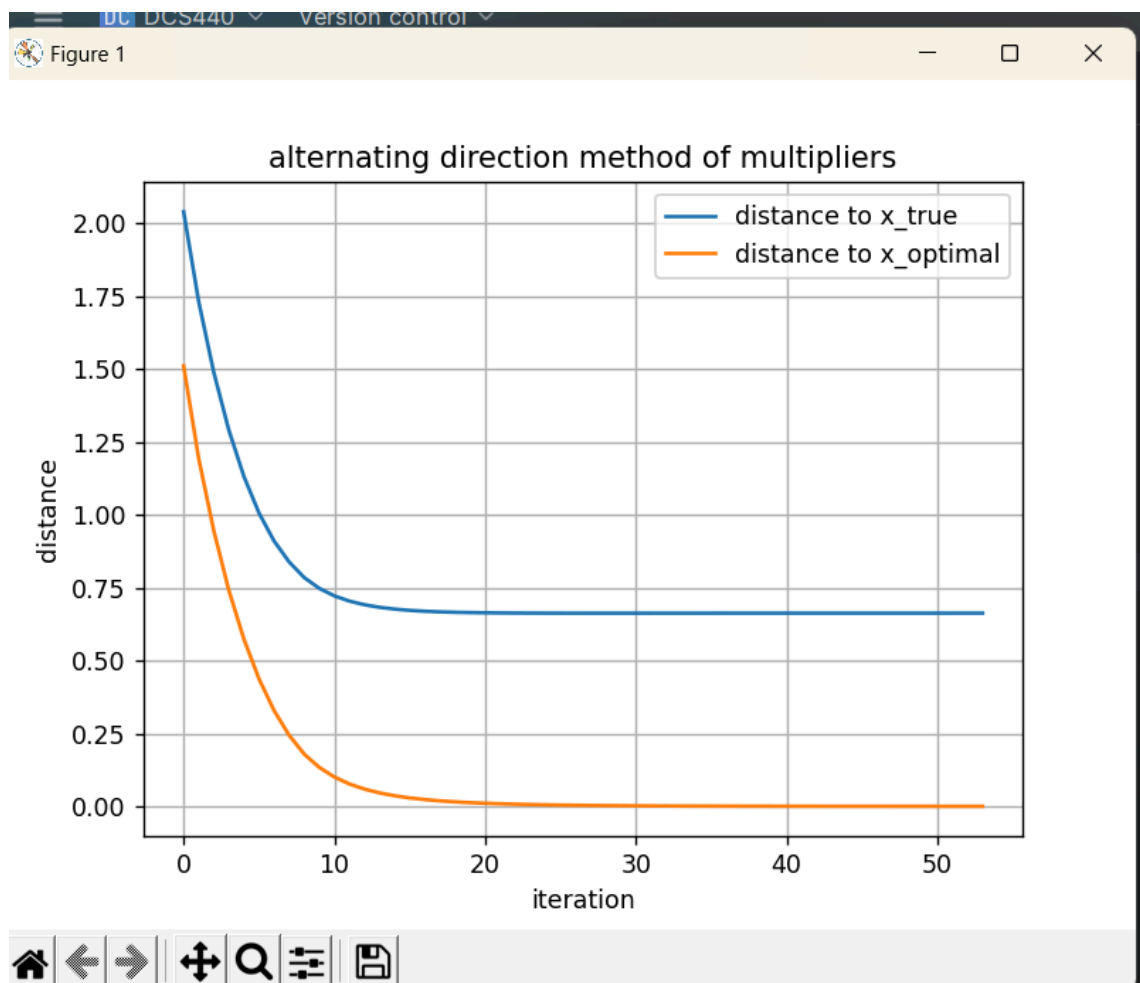
交替方向乘子法

1. $\lambda = 0.001$



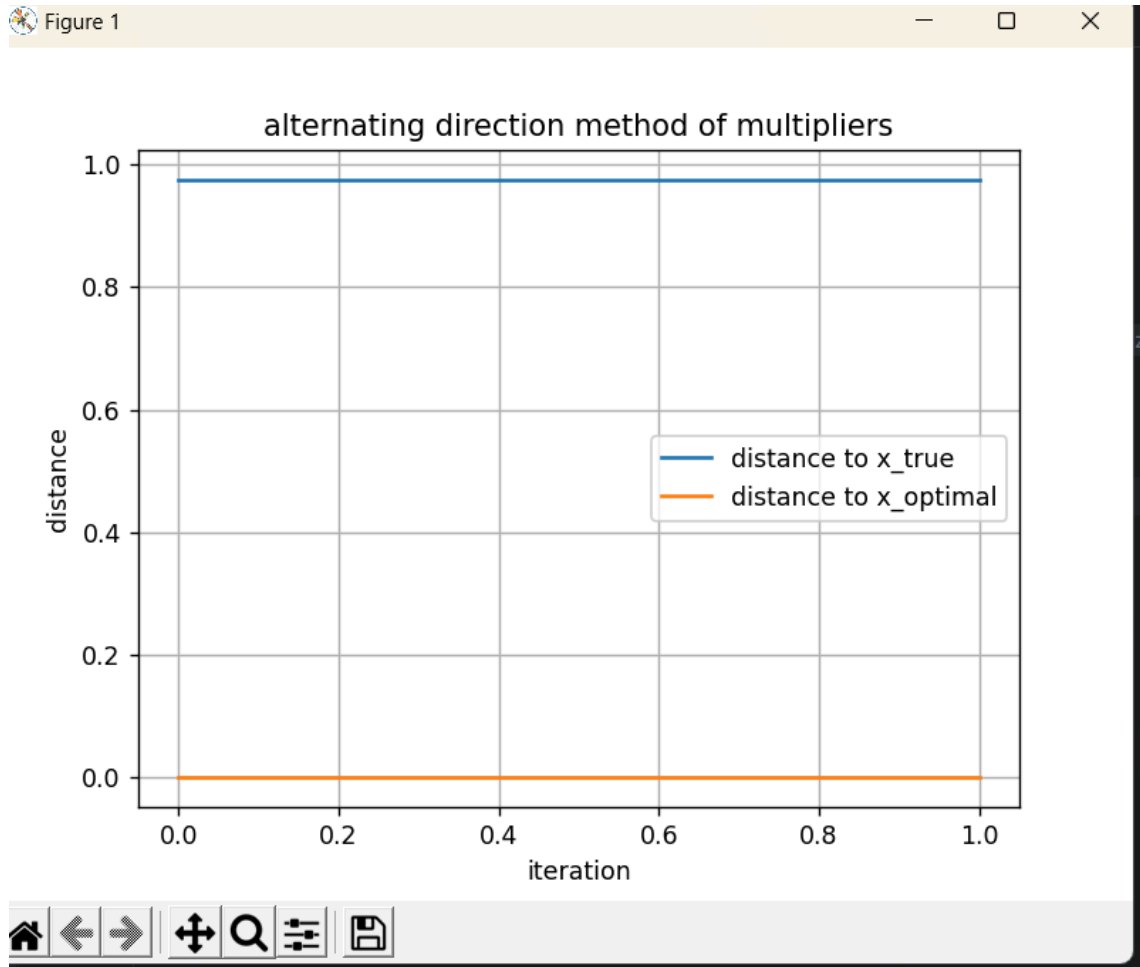
在迭代1750次后收敛，到真值和最优解的距离分别收敛到0.125和0。

2. $\lambda = 0.1$



在迭代50次后收敛，到真值和最优解的距离分别收敛到0.75和0。

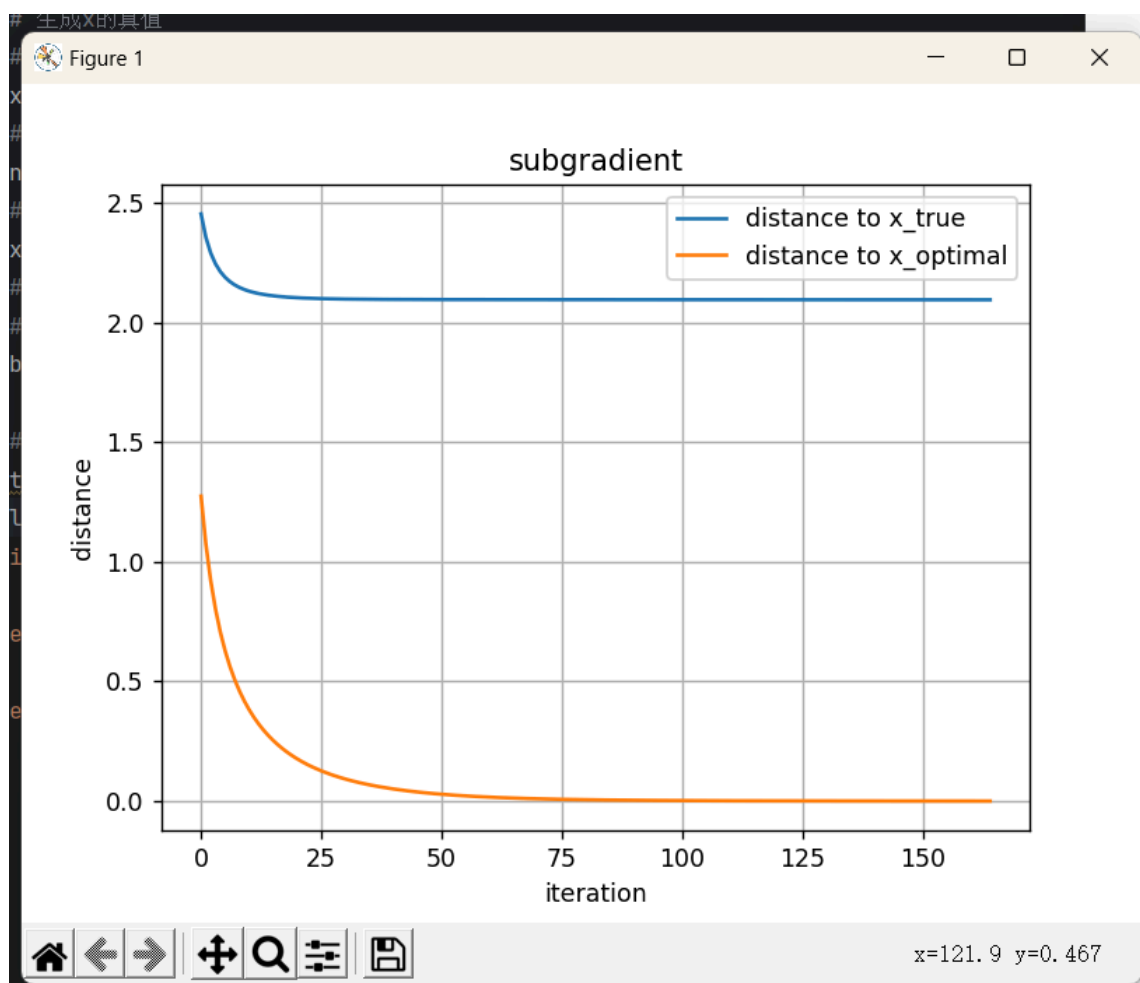
3. $\lambda = 1.0$



对于交替方向乘子法， $\lambda > 0.5$ 后，似乎没有迭代就会收敛。

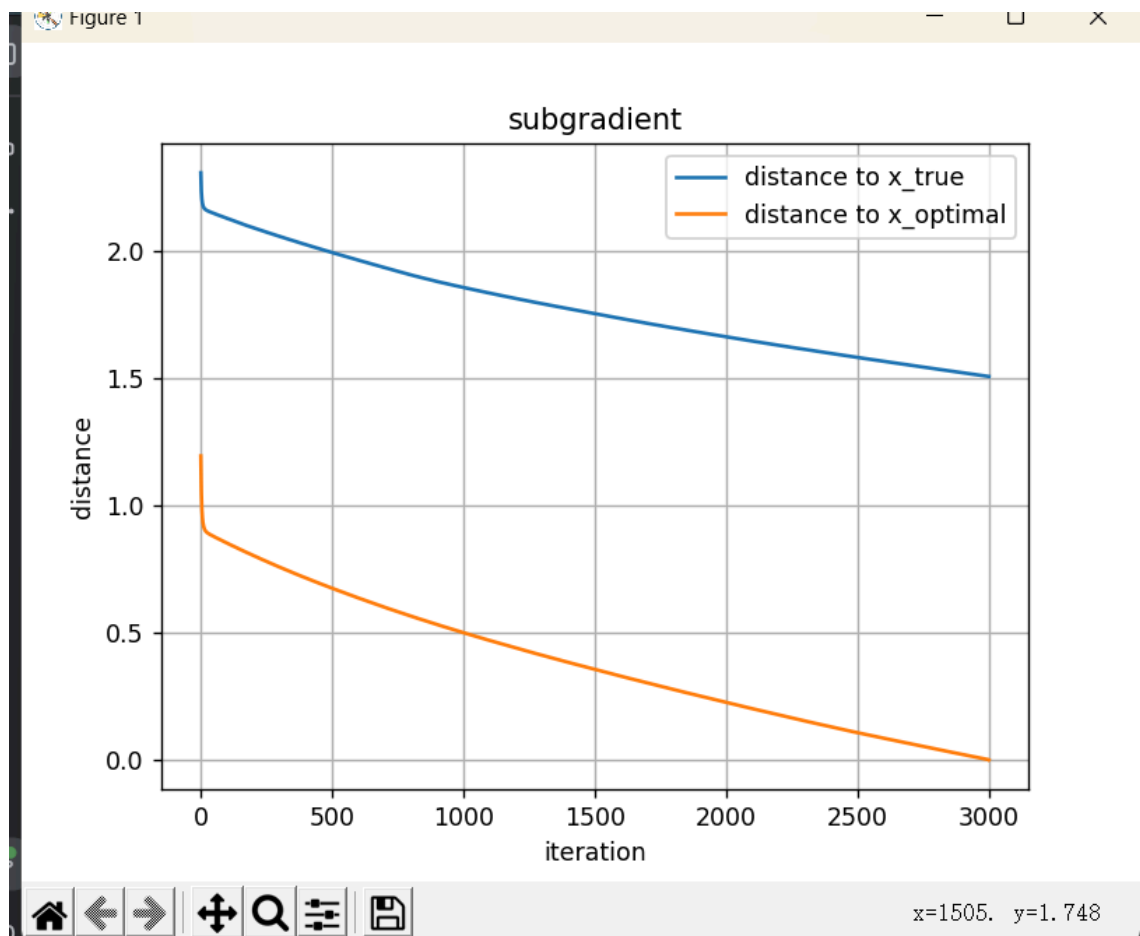
次梯度法

1. $\lambda = 0.001$



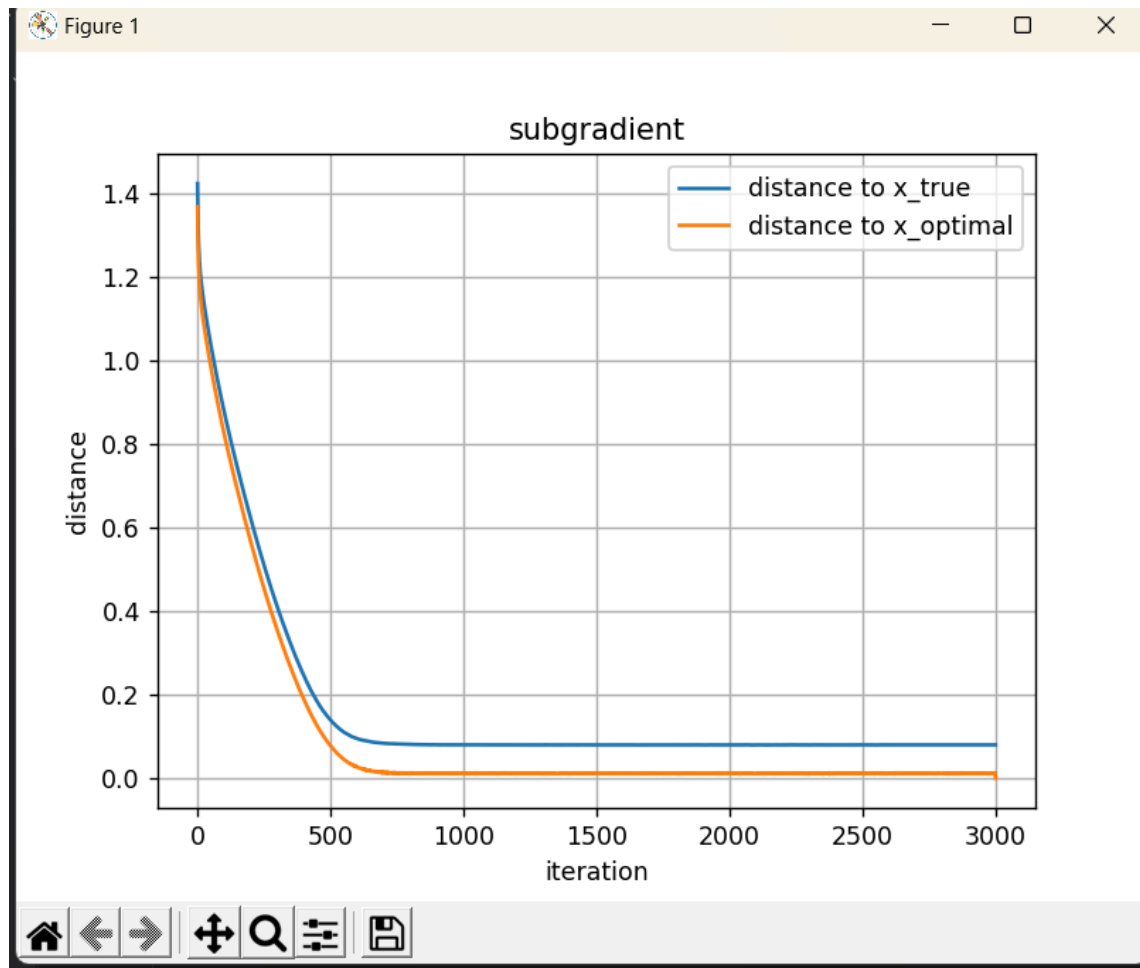
在迭代150次后收敛，到真值和最优解的距离分别收敛到2.1和0。

2. $\lambda = 0.1$



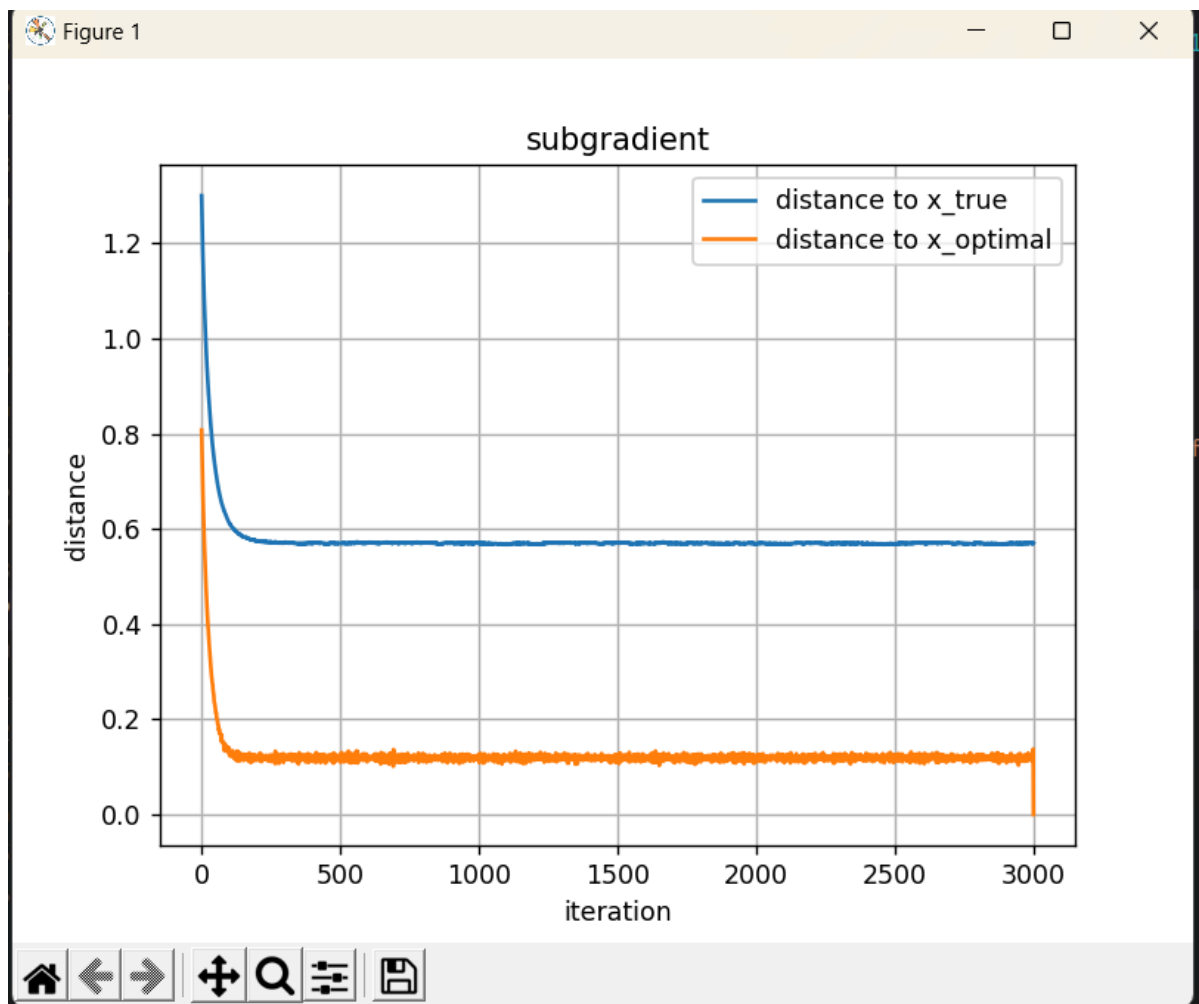
没有收敛，最终到真值和最优解的距离接近1.5和0。

3. $\lambda = 1.0$



没有收敛，最终到真值和最优解的距离为0.1和0.

4. $\lambda = 10.0$



没有收敛，最终到真值和最优解的距离为0.6和0.15.

对于这个问题上的次梯度法，最优的 λ 值在1.0附近。